

Criptografía Básica

El motor invisible detrás de TLS, JWT, firmas y contraseñas.
Simétrica, asimétrica, hashing, KDFs y por qué nunca
inventes la tuya.



- 16 diapositivas
- ~22 min
- Nivel · Iniciación + Práctica

Qué vamos a ver

/01	Por qué importa la criptografía	contexto
/02	Las 4 propiedades · C, I, Aut, NoRep	objetivos
/03	Simétrica · AES, ChaCha20 y modos	una clave
/04	Asimétrica · RSA, ECC, Diffie-Hellman	par de claves
/05	Hashing y MAC	integridad
/06	Firmas y no repudio	autenticidad
/07	Hash de contraseñas · KDFs	argon2 · bcrypt
/08	PKI y certificados X.509	confianza
/09	TLS 1.3 · el handshake	en uso
/10	Errores comunes · y post-cuántica	peligros
/11	Lab · openssl + hashcat	manos a la obra

03 · ESTÁ DEBAJO DE TODO

Cada vez que abres una web HTTPS, validas un JWT, firmas un commit o almacenas una contraseña, **estás usando criptografía**. Saberla leer es lo que separa un buen profesional de uno que copia comandos.

Síntesis a partir de NIST SP 800-175B y la documentación TLS de IETF [1][2]

TLS · tráfico web cifrado

SSH · acceso remoto seguro

VPN · túneles privados

JWT · tokens firmados

PGP · email y firma de paquetes

BitLocker / LUKS · cifrado de disco

Las 4 propiedades que provee

Confidencialidad

Solo quien debe leerlo puede leerlo. El cifrado convierte un mensaje en algo ilegible sin la clave. Provista por *simétrica* y *asimétrica*.

Integridad

No se ha modificado. Hash y MAC detectan cualquier alteración. Sin esta propiedad, el cifrado solo no basta.

Autenticidad

Es de quien dice ser. Lograda con MAC (entre dos partes que comparten clave) o firma digital (clave privada).

No

repudio

No puedes negar haberlo enviado. Sólo lo aporta la firma digital con clave privada (no los MAC).

▲ Cifrar sin autenticar = inseguro. Esquemas modernos usan modos AEAD (AES-GCM, ChaCha20-Poly1305) que combinan ambos en uno.

Criptografía simétrica

Algoritmos // estándar

AES	Bloque de 128 bits · claves 128/192/256. Estándar mundial desde 2001 (FIPS 197).
ChaCha20	Cifrado de flujo moderno · más rápido en software sin acelerador AES.
3DES, RC4, DES	obsoletos · prohibidos en estándares actuales.

Modos de operación // cómo se aplica

GCM (AEAD)	Cifrado + autenticación en uno. Preferido hoy.
CBC + HMAC	Válido si está bien implementado · requiere MAC explícito.
CTR	Convierte un cifrado de bloque en uno de flujo · necesita IV único.
ECB	nunca usar · patrones del plaintext se ven en el cifrado.

Criptografía asimétrica

Cada parte tiene una **privada** (secreta) y una **pública**. Lo que cifra una sólo lo descifra la otra. Resuelve el problema de distribución de claves.

RSA

El clásico (1977). Basado en factorización de enteros grandes. **Mínimo 2048 bits** hoy · ideal 3072+.

ECC

Curva elíptica. Mismo nivel de seguridad con claves mucho más pequeñas (256 bits ECC $\approx$ 3072 RSA). Más rápido.

Diffie-Hellman

Intercambio de claves. Permite a dos partes acordar una clave compartida sobre canal público. Base del TLS handshake.

Ed25519 / ECDSA

Firmas digitales con ECC. Más rápidos y más seguros que RSA para firma. SSH, JWT, Signal, certificados modernos.

▲ Asimétrica es **$\sim 1000\times$ más lenta** que simétrica. Lo habitual: usar asimétrica para acordar una clave simétrica y a partir de ahí, AES/ChaCha.

Hashing y MAC

Funciones hash

// fingerprint

qué hace	Convierte cualquier entrada en una huella de tamaño fijo. One-way .
propiedades	Pre-imagen · 2ª pre-imagen · resistencia a colisiones.
vigentes	SHA-256 · SHA-3 · BLAKE3 .
obsoletos	MD5 y SHA-1 · colisiones públicas desde 2017.

MAC · clave compartida

// integridad + autenticidad

qué es	Una huella que sólo puede generar quien tenga la clave. Verifica origen e integridad.
HMAC	HMAC-SHA-256 es el estándar. Usado en TLS, JWT (HS256), IPsec.
Poly1305	MAC moderno · empareja con ChaCha20 (AEAD).
no aporta	No repudio · ambas partes pueden generar el MAC.

Firmas digitales

El emisor firma con su clave privada el hash del mensaje.
Cualquiera puede verificar con la clave pública. Aporta integridad, autenticidad y **no repudio**.

[01]RSA-PSS · firma RSA con padding probabilístico · más seguro que el viejo PKCS#1 v1.5.	2048+
[02]ECDSA · firma con curva elíptica · habitual en certificados TLS modernos.	P-256+
[03]Ed25519 · firma EdDSA · rápida, determinista, sin riesgos clásicos de aleatoriedad mala.	preferido
[04]PGP / Sigstore · firma de software, releases y artefactos en CI/CD.	supply-chain
[05]JWT con RS256 / ES256 · firma asimétrica de tokens · soporta key rotation.	jwt

Hash de contraseñas · KDFs

argon2id

Ganador de la Password Hashing Competition (2015). Resistente a GPU y a ataques side-channel. Configurable en memoria, paralelismo e iteraciones.

preferido · OWASP recomienda

bcrypt / scrypt

bcrypt es el clásico maduro (cost=12+ hoy). **scrypt** añade memory-hardness. Ambos aceptables.

aceptable · si argon2 no es opción

PBKDF2-SHA-256

Estándar NIST · muchas iteraciones (600k+). Aceptable en entornos regulados, pero menos resistente a GPU.

regulatorio · OK

SHA-256 / MD5 desnudos

Diseñados para ser **rápidos**. Una GPU calcula billones por segundo. Sin salt + KDF, las contraseñas caen en horas.

PROHIBIDOS · para contraseñas

PKI y certificados X.509



▲ Logs públicos · Certificate Transparency (RFC 6962) obliga a las CAs a publicar cada cert emitido. Punto de partida del OSINT moderno.

TLS 1.3 · el handshake

```
// flujo simplificado · 1-RTT
```

```
cliente —▶ ClientHello · key_share (ECDHE)
```

```
server —▶ ServerHello · key_share · {Cert, CertVerify, Finished}
```

```
cliente —▶ {Finished}
```

```
ambos —▶ application_data (cifrado con AES-GCM o ChaCha20-Poly1305)
```

```
// claves: ECDHE genera secreto compartido · KDF deriva claves de sesión · AEAD las usa
```

1-RTT la conexión completa en un solo round-trip · 0-RTT opcional con riesgos.

PFS Forward Secrecy obligatoria · cada sesión usa una clave efímera.

Errores comunes

[01]Inventarse el cifrado · "es secreto, nadie sabrá cómo funciona". Nunca acaba bien.	DIY
[02]ECB · patrones del plaintext visibles. Foto del pingüino de ECB es el ejemplo de manual.	modo
[03]IV/nonce reutilizado · destruye la seguridad de CTR y GCM. Cada cifrado, nuevo IV.	nonce
[04]RNG débil · rand(), Math.random() o tiempo como fuente. Usa /dev/urandom o secrets.	aleatoriedad
[05]Claves embebidas · API keys, claves AES en el binario o en git. Bots las indexan en minutos.	storage
[06]Cifrar sin autenticar · usar AES-CBC sin HMAC → padding oracle attacks (POODLE, BEAST).	AEAD
[07]Algoritmos obsoletos · MD5, SHA-1, DES, RC4, RSA-1024. Listados en CCN-STIC-807.	legacy

Criptografía post-cuántica

Un ordenador cuántico relevante **romperá RSA y ECC con el algoritmo de Shor**. La simétrica (AES, ChaCha) y los hash sólo pierden la mitad del bit-security: aún seguros con tamaños mayores.

ML-KEM

Estándar NIST 2024 · FIPS 203 ·
sustituye al intercambio DH/ECDH
(CRYSTALS-Kyber).

ML-DSA

FIPS 204 · firma digital post-cuántica
(CRYSTALS-Dilithium).

SLH-DSA

FIPS 205 · firma basada en hash
(SPHINCS+) · respaldo conservador.

▲ **Harvest now, decrypt later** · adversarios ya capturan tráfico cifrado para descifrarlo cuando los cuánticos estén listos. Por eso la migración es urgente.

Cifra, firma y rompe un hash débil

```
// crypto  
box
```

Kali Linux

172.X.X.X

· openssl
+ hashcat

sólo local ·
sin red

```
// wordlist
```

rockyou.txt

/usr/share/wordlists

TRES PARTES

- › Cifrado simétrico con **AES-256-GCM**.
- › Generación y firma con **RSA**.
- › Romper un hash **MD5** con diccionario.

```
●●● kali@kali · ~/lab-09 · bash
```

[01] Cifra y descifra con AES-256-CBC:

```
$ echo "secret data" > m.txt  
$ openssl enc -aes-256-cbc -pbkdf2 -in m.txt -out m.enc -k "clave123"  
$ openssl enc -d -aes-256-cbc -pbkdf2 -in m.enc -k "clave123"
```

[02] Genera par RSA-3072 · firma y verifica:

```
$ openssl genrsa -out sk.pem 3072 && openssl rsa -in sk.pem -pubout -  
out pk.pem  
$ openssl dgst -sha256 -sign sk.pem -out m.sig m.txt  
$ openssl dgst -sha256 -verify pk.pem -signature m.sig m.txt
```

[03] Rompe un hash MD5 con john:

```
$ echo -n "password123" | md5sum | awk '{print $1}' > hash.txt  
$ gunzip -c /usr/share/wordlists/rockyou.txt.gz > /tmp/rockyou.txt  
$ john --format=raw-md5 --wordlist=/tmp/rockyou.txt hash.txt  
$ john --show --format=raw-md5 hash.txt
```

6 ideas para llevarte

- /01 Cuatro propiedades · **confidencialidad, integridad, autenticidad y no repudio**. Cada una pide su herramienta.
- /02 **Simétrica** (AES, ChaCha20) para el grueso del cifrado · **asimétrica** (RSA, ECC) para acordar claves y firmar.
- /03 Para contraseñas, **KDF lenta** con salt: argon2id/bcrypt. Nunca SHA-256 desnudo.
- /04 La PKI traduce "clave pública" en "identidad verificable" via **certificados X.509 + CAs**.
- /05 TLS 1.3 cierra la conexión en **1-RTT con Forward Secrecy obligatoria**.
- /06 No inventes criptografía. Usa AEAD (GCM, ChaCha20-Poly1305), librerías auditadas y planifica **post-cuántica**.

SIGUIENTE · TEMA 10

Seguridad en la Nube

El modelo de responsabilidad compartida, IAM, redes virtuales, gestión de secretos y los riesgos más comunes en AWS, Azure y GCP.

→ continúa en el siguiente vídeo

Para profundizar

NIST e IETF marcan los estándares vigentes. OWASP y CCN-CERT dan guías aplicadas.

[1]	NIST	FIPS 197 (AES) · 180-4/202 (SHA) · 186-5 (firma) · SP 800-38, 800-56, 800-63B, 800-131A, 800-175B
[2]	IETF	RFC 8446 (TLS 1.3) · 5280 (X.509) · 8032 (EdDSA) · 7539 (ChaCha20) · 9106 (Argon2)
[3]	OWASP	Cryptographic Storage · Password Storage · Transport Layer Protection · owasp.org
[4]	NIST PQC	FIPS 203 (ML-KEM) · 204 (ML-DSA) · 205 (SLH-DSA) · csrc.nist.gov/projects/pqc
[5]	CCN-CERT / ENISA	CCN-STIC-807 · informes PQC · ccn-cert.cni.es · enisa.europa.eu
[6]	Bibliografía	Katz & Lindell — <i>Intro to Modern Cryptography</i> · Schneier — <i>Applied Cryptography</i>